National Institute of Business Management

School of Computing and Engineering

Project Report

Database Management System

HNDNE 25.2F

Group Memebers

Index No	Name
COHNDNE252F-001	FERNANDO M V S
COHNDNE252F-003	METHNULI J K P D D

STATEMENT OF AUTHENTICITY

Declaration of Work Originality

I hereby declare that the coursework report named “Design and Implementation of Secure MySQL Database” is an original piece of work done by our team for the module on Database Security course at National Institute of Business Management (NIBM).

All the work done in this report has been done by us with our independent practical implementation, research, and analysis. All other external sources, references, technicalities, and documents involved in this report have been appropriately mentioned.

We hereby declare that this report is consistent with the university’s guidelines regarding plagiarism and academic integrity.

Group Members:

1. _____ J K P D D Methnuli
2. _____ M V S Fernando

Date: 21/05/2026

WORD OF ACKNOWLEDGMENT

It gives us great pleasure to convey our heartfelt thanks to our lecturer, **Mr. Gayan Sampath**, for his unwavering guidance and encouragement during this Database Security coursework project.

The knowledge and technical skills of **Mr. Gayan** have proved very helpful to us, especially when it came to grasping key concepts in database security, replication, access control, high availability, and security auditing. His guidance played an instrumental role in our successful completion of this project.

We are thankful to the National Institute of Business Management (NIBM) for having provided us with the required environment and facilities to conduct this practical project.

Lastly, we thank each and every member of the team for their contribution towards the success of this project.

TABLE OF CONTENT

1. Introduction…….. **04**
2. System Architecture Design…… **05**
3. ILO1 — SECURE DATABASE DESIGN AND CIA TRIAD MAPPING…… **06**
4. ILO 2 — AUTHENTICATION AND ACCESS SECURITY IMPLEMENTATION……. **12**
5. ILO 3 — DATABASE ACCESS CONTROL MODELS IMPLEMENTATION……. **17**
6. ILO 4 — THREAT IDENTIFICATION AND MITIGATION…….. **22**
7. ILO 5 — HIGH AVAILABILITY AND READ SCALABILITY IMPLEMENTATION…….…… **30**
8. ILO 6 — INDEPENDENT DATABASE SECURITY AUDIT AND COMPLIANCE REVIEW…….…… **36**
9. Final Exploration…… **39**
10. References …….. **40**
11. Appendix……. **41**

INTRODUCTION

The process of safeguarding databases from the unlawful access, misuse, attacks and loss of data. It is often aligned with the core concept of CIA TRIAD in order to ensure that the confidentiality, integrity and availability is protected for secure and high available purposes.

The assignment focuses on implementing and designing a secure MYSQL Database infrastructure for a financial services company.

The practical implementation includes of Customer records, Employee Information, Transaction records, Authentication data, Audit logs and Backup records.

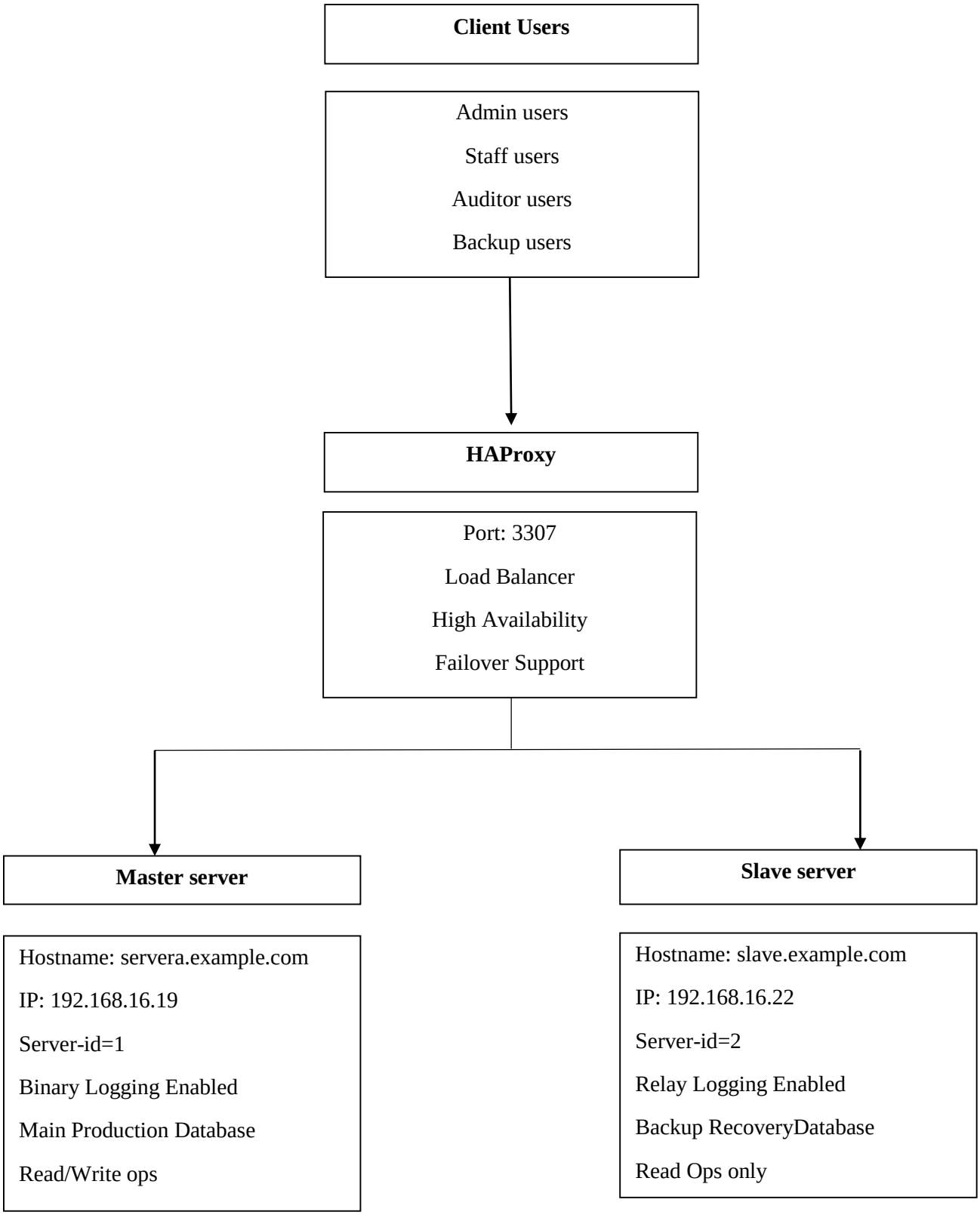
Goals to be achieved:

- Secure Database Design
- Authentication Implementations
- High Availability
- Access control mechanisms
- Threat mitigation
- MySQL replication
- Database auditing and compliance review

Technologies Used:

- MySQL Server
- Linux/RHEL
- HAProxy
- FirewallD

SYSTEM ARCHITECTURE DESIGN



ILO1 - SECURE DATABASE DESIGN AND CIA TRIAD MAPPING

Database Creation

The process of initializing and configuring a structured environment for secure organizational data storage**.**

Commands

```
mysql> CREATE DATABASE enterprise_secure_db;
Query OK, 1 row affected (0.00 sec)
```

Clarification

Enterprise secure database is developed for the purpose of ensuring that the company information is secure, which include customer information, employee information, security information, financial transactions, audit trail and backups. Information like identity numbers, salaries information, login passwords, and customer contact details are firmly secured in the database environment.

Table Creation

Commands

```
Database changed
mysql> CREATE TABLE users (
    ->     user_id INT PRIMARY KEY AUTO_INCREMENT,
    ->     username VARCHAR(50) NOT NULL UNIQUE,
    ->     password_hash VARCHAR(255) NOT NULL,
    ->     role ENUM('admin','staff','auditor','backup') NOT NULL,
    ->     email VARCHAR(100) NOT NULL UNIQUE,
    ->     phone_number VARCHAR(15),
    ->     account_status ENUM('active','inactive','locked') DEFAULT 'active',
    ->     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    ->
```

```
mysql> CREATE TABLE employees (
    ->     employee_id INT PRIMARY KEY AUTO_INCREMENT,
    ->     user_id INT,
    ->     full_name VARCHAR(100) NOT NULL COMMENT,
    ->     national_id VARCHAR(20) UNIQUE COMMENT 'PII - National ID Number',
    ->     salary DECIMAL(10,2) ,
    ->     classification_level VARCHAR(20) DEFAULT 'Confidential',
    ->     FOREIGN KEY (user_id)
    ->     REFERENCES users(user_id)
    -> );
```

```
mysql> CREATE TABLE customers (
    ->     customer_id INT PRIMARY KEY AUTO_INCREMENT,
    ->     full_name VARCHAR(100) NOT NULL COMMENT 'PII',
    ->     national_id VARCHAR(20) UNIQUE COMMENT 'PII',
    ->     email VARCHAR(100) COMMENT 'PII',
    ->     phone VARCHAR(15) COMMENT 'PII',
    ->     classification_level VARCHAR(20) DEFAULT 'Confidential'
    -> );
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> CREATE TABLE transactions (
    ->     transaction_id INT PRIMARY KEY AUTO_INCREMENT,
    ->     customer_id INT,
    ->     amount DECIMAL(10,2) COMMENT 'Financial',
    ->     classification_level VARCHAR(20) DEFAULT 'Restricted',
    ->     transaction_date DATE,
    ->
    ->     FOREIGN KEY (customer_id)
    ->     REFERENCES customers(customer_id)
    -> );
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> CREATE TABLE audit_logs (
    ->     log_id INT PRIMARY KEY AUTO_INCREMENT,
    ->     user_id INT,
    ->     action VARCHAR(100),
    ->     classification_level VARCHAR(20) DEFAULT 'Internal',
    ->     log_date DATE,
    ->
    ->     FOREIGN KEY (user_id)
    ->     REFERENCES users(user_id)
    -> );
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> CREATE TABLE backup_records (
    ->     backup_id INT PRIMARY KEY AUTO_INCREMENT,
    ->     backup_name VARCHAR(100) COMMENT 'Internal',
    ->     classification_level VARCHAR(20) DEFAULT 'Internal',
    ->     backup_date DATE,
    ->     created_by INT,
    ->
    ->     FOREIGN KEY (created_by)
    ->     REFERENCES users(user_id)
    -> );
Query OK, 0 rows affected (0.01 sec)
```

Clarification

Multiple interconnected tables are created so that information in the organization can be organized and managed effectively. These include database structures for managing customers, employees, transactions, authentication management, auditing, and backups.

Primary Key Implementation

Primary key – identifies each records within the table uniquely. Pure purpose of using PK is to ensure no duplication complications are found while implementing record entries.

Visual proof:

```
 user_id INT PRIMARY KEY AUTO_INCREMENT,
```

```
employee_id INT PRIMARY KEY AUTO_INCREMENT,
```

```
 customer_id INT PRIMARY KEY AUTO_INCREMENT,
```

```
transaction_id INT PRIMARY KEY AUTO_INCREMENT,
```

```
log_id INT PRIMARY KEY AUTO_INCREMENT,
```

```
backup_id INT PRIMARY KEY AUTO_INCREMENT,
```

Foreign Key Implementation

Visual proof:

```
FOREIGN KEY (user_id)
REFERENCES users(user_id)
```

```
FOREIGN KEY (customer_id)
REFERENCES customers(customer_id)
```

```
FOREIGN KEY (user_id)
REFERENCES users(user_id)
```

```
FOREIGN KEY (created_by)
REFERENCES users(user_id)
```

Clarification

Foreign Keys – establish the relationships with interconnected database tables. These relationships are being constructed to uphold referential integrity and ensure dependability and stability between tables that are related.

Constraints and Validation

Visual proof:

```
username VARCHAR(50) NOT NULL UNIQUE,
password_hash VARCHAR(255) NOT NULL,
role ENUM('admin','staff','auditor','backup') NOT NULL,
email VARCHAR(100) NOT NULL UNIQUE,
```

```
full_name VARCHAR(100) NOT NULL COMMENT,
national_id VARCHAR(20) UNIQUE COMMENT 'PII - National ID Number',
```

```
customer_id INT PRIMARY KEY AUTO_INCREMENT,
full_name VARCHAR(100) NOT NULL COMMENT 'PII',
national_id VARCHAR(20) UNIQUE COMMENT 'PII',
email VARCHAR(100) COMMENT 'PII',
phone VARCHAR(15) COMMENT 'PII',
classification_level VARCHAR(20) DEFAULT 'Confidential'
```

```
transaction_id INT PRIMARY KEY AUTO_INCREMENT,
customer_id INT,
amount DECIMAL(10,2) COMMENT 'Financial',
classification_level VARCHAR(20) DEFAULT 'Restricted',
```

```
log_id INT PRIMARY KEY AUTO_INCREMENT,
user_id INT,
action VARCHAR(100),
classification_level VARCHAR(20) DEFAULT 'Internal'
```

```
backup_id INT PRIMARY KEY AUTO_INCREMENT,
backup_name VARCHAR(100) COMMENT 'Internal',
classification_level VARCHAR(20) DEFAULT 'Internal'
```

Clarification

These mechanisms constraints and validation, impose rules in order to maintain precise and dependable and trustworthy data storage. Constraints like – UNIQUE and NOT NULL guarantee validation of rules, duplicate record complication prevention, mandatory records are being stored appropriately.

CIA TRIAD MAPPING

CONFIDENTIALITY

Definition: Ensure that sensitive information is accessible only to the users who are authorized.

Controls Implemented:

- User authentication
- Password protection
- Privilege restrictions
- Role separation

Clarification:

Confidentiality was achieved through the use of MySQL authentication mechanisms, separation of privileges, and access control mechanisms that ensured the security of sensitive organization information such as salary details, customer contact information, authentication details, and transaction details.

INTEGRITY

Definition: Intefgrity make certain that the records stay accurate consistent and not modified unless it is under the supervision of authorized users.

Controls Implemented:

- Primary keys
- Foreign keys
- Constraints
- Validation mechanisms

Clarification:

Integrity is applied through the usage of primary key, foreign key relationships, constraints and validation mechanisms in order to dismiss the complications that can be arisen with duplication entries, invalid relationships and further the data modifications that are not consistent.

AVAILABILITY

Definition: availability is to guarantee that a required service is accessible and functions operational wise whenever needed.

Controls Implemented:

- Database backups
- MySQL replication
- HAProxy high availability architecture

Clarification:

In order to improve the database architecture in terms availability it is applied with mechanisms and procedures like MySQL replication, HAProxy load balancing architecture to confirm in case of traffic overload the reduced downtime of the system and in addition enhanced disaster recovery capabilities.

ILO 2 —AUTHENTICATION AND ACCESS SECURITY IMPLEMENTATION

User Creation:

User account is created to oblige with the requirements of privilege escalation and control access management

Implementation of Authentication Methods

- mysql_native_password

```
mysql> CREATE USER 'staff_user'@'%' IDENTIFIED WITH mysql_native_password BY 'Staff@123';
Query OK, 0 rows affected (0.00 sec)

mysql> CREATE USER 'auditor_user'@'%' IDENTIFIED WITH caching_sha2_password BY 'Audit@123';
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE USER 'backup_user'@'%' IDENTIFIED WITH mysql_native_password BY 'Backup@123';
Query OK, 0 rows affected (0.00 sec)
```

A MySQL authentication plugin – uses the method of password hashing for the authentication and for the purpose of replication compatibility.

- caching_sha2_password

```
mysql> CREATE USER 'admin_user'@'%'
    -> IDENTIFIED WITH caching_sha2_password BY 'Admin@123';
Query OK, 0 rows affected (0.01 sec)
```

Uses SHA-256 encryption and the particular method provide stronger password protection and enhanced security for AuthN procedures.

AUTHENTICATION CONCEPTS

- PAM
- Kerberos

PAM Authentication Concept

PAM (Pluggable Authentication Modules) - Allows MySQL in order to incorporate with LINUX authentication systems. Further it validates MySQL users against the Operating System accounts by permitting the settings via configurations for the centralized Authentication Management.

Kerberos Authentication Concept

KerberOS is a network authentication protocol used for secure identity verification used to securely verify the identity of users and devices over a network. Basically, Ticket based and secret key cryptography, so the passwords are not required to sent over the network again and gain repeatedly.

Goals of KerberOS:

- Prevent Password theft
- Provide secure login
- User authentication

Security Hardening (disable anonymous users, root restrictions, password policy)

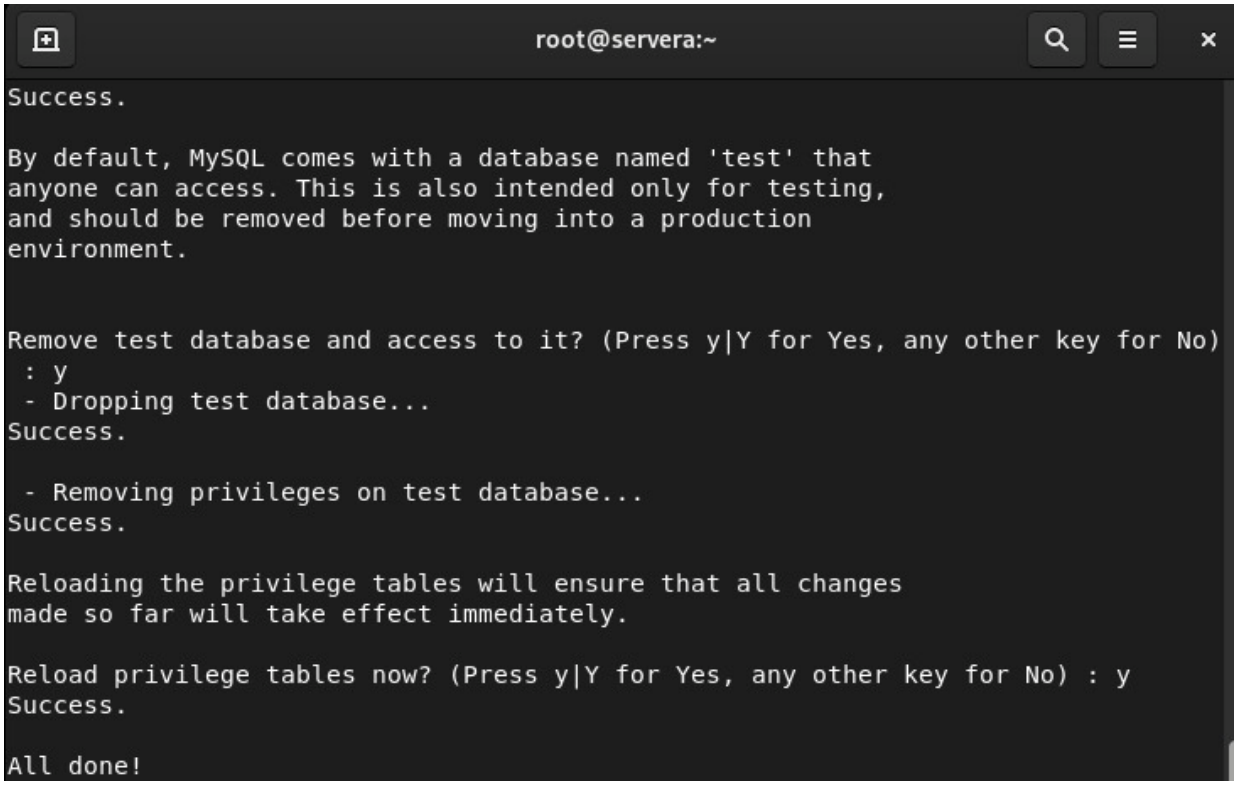
Intended Hardening Measures to Be Implemented:

- Remove anonymous users
- Remove test database
- Enable password validation
- Restricte insecure access
- Configure firewall security

Visual Proof:







Clarification:

The above procedure is implemented in order to amplify the level of security. In addition these procedures reduce the risk of the vulnerabilities and strengthen and consolidate authentication security.

Authentication Comparison

Method Used	Security Level	Use cases
mysql_native_password	Medium	• Mostly for compatible with older systems • Outdated • Used for legacy systems
caching_sha2_password	High	• Modern method • More secure • Used for authentication processes
PAM	High	• Linux authentication Integration
KerberOS	Extremely High	• Used Enterprise Level • Enables Single Sign On (SSO)

ILO 3 — DATABASE ACCESS CONTROL MODELS IMPLEMENTATION

DAC Implementation

Commands:

```
mysql> GRANT SELECT, INSERT ON enterprise_secure_db.* TO 'staff_user'@'%';
Query OK, 0 rows affected (0.00 sec)

mysql> SHOW GRANTS FOR 'staff_user'@'%';
+-------------------------------------------------------------------------+
| Grants for staff_user@%                                                 |
+-------------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `staff_user`@`%`                                  |
| GRANT SELECT, INSERT ON `enterprise_secure_db`.* TO `staff_user`@`%` |
| GRANT `staff_role`@`%` TO `staff_user`@`%`                              |
+-------------------------------------------------------------------------+
3 rows in set (0.00 sec)
```

```
mysql> GRANT SELECT ON enterprise_secure_db.* TO 'auditor_user'@'%';
Query OK, 0 rows affected (0.00 sec)
```

Explanation:

Discretionary Access Control – basically the owner of a particular source or resource has the complete authority to decide on who gets the access. It's easy to manage and in addition, in terms of flexibility sharing is simple. But there are some drawbacks within DAC, it's less secure due to accidental issues, malware being able to misuse permissions.

MAC Simulation

Classification Level:

- Public
- Internal
- Confidential
- Restricted

Evidence for Restricted

```
[root@servera ~]# mysql -uauditor_user -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 128
Server version: 8.0.45 Source distribution

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE enterprise_secure_db
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A
```

```
mysql> SELECT * FROM transactions WHERE classification_level = 'Restricted';
+----------------+-------------+----------+----------------------+------------------+
| transaction_id | customer_id | amount   | classification_level | transaction_date |
+----------------+-------------+----------+----------------------+------------------+
|              1 |           1 |  5000.00 | Restricted           | 2026-05-01       |
|              2 |           2 | 12000.00 | Restricted           | 2026-05-02       |
|              3 |           3 |  7500.00 | Restricted           | 2026-05-03       |
|              4 |           4 | 20000.00 | Restricted           | 2026-05-04       |
|              5 |           5 |  3000.00 | Restricted           | 2026-05-05       |
+----------------+-------------+----------+----------------------+------------------+
5 rows in set (0.00 sec)
```

Evidence for Internal

```
[root@servera ~]# mysql -ustaff_user -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 121
Server version: 8.0.45 Source distribution

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> SELECT * FROM employees WHERE classification_level = 'Internal';
```

```
mysql> USE enterprise_secure_db
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SELECT * FROM employees WHERE classification_level = 'Internal';
+-------------+---------+----------------+-------------+----------+----------------------+
| employee_id | user_id | full_name      | national_id | salary   | classification_level |
+-------------+---------+----------------+-------------+----------+----------------------+
|           5 |       5 | Ravi Jayasinghe | NIC005      | 80000.00 | Internal             |
+-------------+---------+----------------+-------------+----------+----------------------+
1 row in set (0.00 sec)
```

Evidence for Confidential

```
Database changed
mysql> SELECT * FROM customers WHERE classification_level IN ('Internal','Confidential');
+-------------+------------------+-------------+--------------------+------------+----------------------+
| customer_id | full_name        | national_id | email              | phone      | classification_level |
+-------------+------------------+-------------+--------------------+------------+----------------------+
|           1 | Amal Perera      | CUST001     | amal@gmail.com     | 0771111111 | Confidential         |
|           2 | Nimal Dias       | CUST002     | nimal@gmail.com    | 0772222222 | Confidential         |
|           3 | Kasun Silva      | CUST003     | kasun@gmail.com    | 0773333333 | Confidential         |
|           4 | Sanduni Fernando | CUST004     | sanduni@gmail.com  | 0774444444 | Confidential         |
|           5 | Tharindu Jay     | CUST005     | tharindu@gmail.com | 0775555555 | Confidential         |
+-------------+------------------+-------------+--------------------+------------+----------------------+
5 rows in set (0.00 sec)
```

Explanation:

The access is controlled by a central authority or a system. Permissions can't be changed by users. This is way more secure and highly strict compared to DAC. The low level user cannot access higher-level data. But it's harder to manage and less flexible.

RBAC Implementation

Commands:

```
mysql> CREATE ROLE 'admin_role';
Query OK, 0 rows affected (0.00 sec)

mysql> GRANT ALL PRIVILEGES ON enterprise_secure_db.* TO 'admin_role';
Query OK, 0 rows affected (0.00 sec)

mysql> CREATE ROLE 'staff_role';
Query OK, 0 rows affected (0.00 sec)

mysql> GRANT SELECT, INSERT ON enterprise_secure_db.* TO 'staff_role';
Query OK, 0 rows affected (0.00 sec)

mysql> CREATE ROLE 'auditor_role';
Query OK, 0 rows affected (0.00 sec)

mysql> CREATE ROLE 'backup_role';
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> GRANT 'admin_role' TO 'admin_user'@'%';
Query OK, 0 rows affected (0.00 sec)

mysql> GRANT 'staff_role' TO 'staff_user'@'%';
Query OK, 0 rows affected (0.00 sec)

mysql> GRANT 'auditor_role' TO 'auditor_user'@'%';
Query OK, 0 rows affected (0.00 sec)

mysql> GRANT 'backup_role' TO 'backup_user'@'%';
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> SET DEFAULT ROLE ALL TO 'admin_user'@'%';
Query OK, 0 rows affected (0.01 sec)

mysql> SET DEFAULT ROLE ALL TO 'staff_user'@'%';
Query OK, 0 rows affected (0.00 sec)

mysql> SET DEFAULT ROLE ALL TO 'auditor_user'@'%';
Query OK, 0 rows affected (0.00 sec)

mysql> SET DEFAULT ROLE ALL TO 'backup_user'@'%';
Query OK, 0 rows affected (0.00 sec)
```

Explanation:

Role Based Access Control - Here the access is totally depending on the concept of user roles. Permissions are appointed according to the role the user upheld with within the organization, not individual based. This RBAC implementation is easy for the organization where the permission division is required based on the role the employees are assigned with. Better concept that aligns with requirements of permission management systems.

Access Control Comparison

Feature	DAC	MAC	RBAC
Controlled By	Resource Owner	System Authority	User Role
Flexibility-level	High	Low	Medium
Security-Level	Lower	Low	High
Users Can Change Permission	Yes	No	Usually No
Use-cases	Personal Systems	Mostly Military Government based	For organizations and companies
Example	File-sharing	Classified data	Employee Systems

ILO 4 — THREAT IDENTIFICATION AND MITIGATION

Selected Threat Scenario – SQL Injection Attack

SQL injection is a threat that involves compromising databases through the injection of malicious SQL code into input fields of applications. This vulnerability occurs because of failure to sanitize user inputs that are then added to the SQL code.

Some of the effects of an SQL Injection attack include:

- Bypassing login authentication
- Accessing sensitive information
- Deleting or modifying data
- Gain unauthorized access to systems

Database Vulnerability Implementation

- A database titled test_security has been developed in order to implement a sample login application susceptible to SQL injection attacks.

```
mysql> CREATE DATABASE test_security;
Query OK, 1 row affected (0.01 sec)
```

- Test_security database has a table named 'login_users' created to record the login details of users for use in simulating a normal login process.

```
mysql> USE test_security
Database changed
mysql> CREATE TABLE login_users (
    ->     id INT AUTO_INCREMENT PRIMARY KEY,
    ->     username VARCHAR(50),
    ->     password VARCHAR(50)
    -> );
Query OK, 0 rows affected (0.01 sec)
```

- Using the INSERT INTO command some records were added to the login_users table.

  These sample accounts are used to test login functionality and demonstrate how SQL Injection can bypass authentication.

  In this vulnerable implementation:

  - → Passwords are stored in plain text
  - → No password hashing is used
  - → The system becomes more vulnerable if attackers access the database

```
mysql> INSERT INTO login_users (username, password)
    -> VALUES
    -> ('admin', 'admin123'),
    -> ('user1', 'pass123');
Query OK, 2 rows affected (0.00 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

Vulnerable Login Query

- The SELECT statement retrieves records from the login_users table. And the query is designed to authenticate users during login.

  If a matching username and password are found:

  - → The user is considered authenticated
  - → Access to the system is granted

```
mysql> SELECT * FROM login_users
    -> WHERE username = 'admin'
    -> AND password = 'admin123';
+----+----------+----------+
| id | username | password |
+----+----------+----------+
|  1 | admin    | admin123 |
+----+----------+----------+
1 row in set (0.00 sec)
```

Why the Query is Vulnerable

The application directly places user input into the SQL statement without protection.

This allows attackers to:

- Manipulate the SQL query
- Inject additional SQL conditions
- Bypass authentication checks

The database interprets malicious input as part of the SQL command.

SQL Injection Attack Demonstration

- The original query structure is modified by the attacker's input and the attacker insert and injected condition **'1'='1'** which always evaluates to TRUE.
  - → The query returns records
  - → Authentication is bypassed
  - → Unauthorized access may be granted

```
mysql> SELECT * FROM login_users
    -> WHERE username = 'admin'
    -> AND password = '' OR '1'='1';
+----+----------+----------+
| id | username | password |
+----+----------+----------+
|  1 | admin    | admin123 |
|  2 | user1    | pass123  |
+----+----------+----------+
2 rows in set (0.00 sec)
```

How the Attack Works

The SQL Injection attack works because:

1. The application accepts unsanitized user input
2. User input is directly inserted into SQL queries
3. The database executes the manipulated query

Instead of treating input as normal text, the database interprets it as SQL code.

As a result:

- → Authentication logic is broken
- → Security checks fail
- → Attackers may gain unauthorized access

Business Impact

- Unauthorized access

  SQL injection helps the attackers to bypass the entire login process of the system.

  Impacts

  - → Unauthorized users may gain access to restricted system areas
  - → Administrative privileges may be obtained by attackers
  - → Sensitive system settings can be modified
  - → Attackers may create fake or unauthorized accounts
  - → Overall system security becomes compromised

- Data Breach

  Attackers may retrieve confidential information stored inside the database through SQL Injection vulnerabilities.

  Examples of Exposed Data

  - → Usernames and passwords
  - → Customer personal information
  - → Financial and banking details
  - → Employee records
  - → Business transaction information

  Impacts

  - → Loss of confidentiality of sensitive data
  - → Exposure of customer privacy
  - → Increased risk of fraud and identity theft
  - → Violation of data protection regulations
  - → Legal penalties and compliance issues

- Data Manipulation

  Attackers may modify, insert, or delete database records using malicious SQL commands.

  Examples
  - → Changing customer details
  - → Modifying financial records
  - → Altering product prices
  - → Deleting important data

  Impacts
  - → Corruption of database information
  - → Loss of data integrity and accuracy
  - → Incorrect reports and business decisions
  - → Disruption of business operations
  - → Difficulty recovering original data

- Financial Loss

  Organizations may experience significant financial damage after a successful SQL Injection attack.

  Impacts
  - → Costs for system recovery and repairs
  - → Expenses for database restoration
  - → Operational downtime and service interruptions
  - → Loss of business revenue
  - → Regulatory fines and legal costs
  - → Compensation payments to affected customers

- Reputational Damage

A security breach can negatively affect customer trust and the organization's public image.

Impacts

- → Loss of customer confidence
- → Negative publicity and media attention
- → Damage to organizational reputation
- → Reduction in customer base
- → Difficulty attracting new customers and business partners

Mitigation Implementation

- The Principle of Least Privilege

The principle of least privilege refers to that users of the database should only have the privileges they require to perform their tasks.
This helps to minimize the effect of SQL injection attacks.

1. Creating a restricted database user. Using a separate restricted account is safer than using the MySQL root account.

```
mysql> CREATE USER 'secure_user'@'localhost'
    -> IDENTIFIED BY 'StrongPass123!';
Query OK, 0 rows affected (0.07 sec)
```

2. Granting Limited permissions that is required for the user

   The user can:

   - → Read records

   The user cannot:

   - → Delete records
   - → Modify database structure
   - → Drop tables
   - → Create users

   This limits the damage attackers can cause.

- Password Hashing

A method of security that encrypts password into a fixed-size string that cannot be decrypted back to the original password in the system is called password hashing.

The passcode is stored as a hash function that translates the string of characters into a unique number — for example, using the SHA256 hash function — rather than storing it in plain text as a series of characters (e.g., admin123). Insert Hashed Password

In here the SHA2() hashing function converts the password into a secure hash value before storing it in the database.

Hashing protects passwords because:

→ Original passwords are not stored directly

→ Attackers cannot easily read passwords

```
mysql> INSERT INTO login_users(username, password)
    -> VALUES
    -> ('admin', SHA2('admin123',256));
Query OK, 1 row affected (0.00 sec)
```

1. Verify Hashed Password

The user entered login password is hashed again using SHA2() function and compared with the stored hash value.

If both hashes match, the authentication process succeeds and this method helps improve the password security.

```
mysql> SELECT *
    -> FROM login_users
    -> WHERE username='admin'
    -> AND password=SHA2('admin123',256);
+----+----------+------------------------------------------------------------------+
| id | username | password                                                         |
+----+----------+------------------------------------------------------------------+
|  5 | admin    | 240be518fabd2724ddb6f04eeb1da5967448d7e831c08c8fa822809f74c720a9 |
+----+----------+------------------------------------------------------------------+
1 row in set (0.00 sec)
```

Clarification:

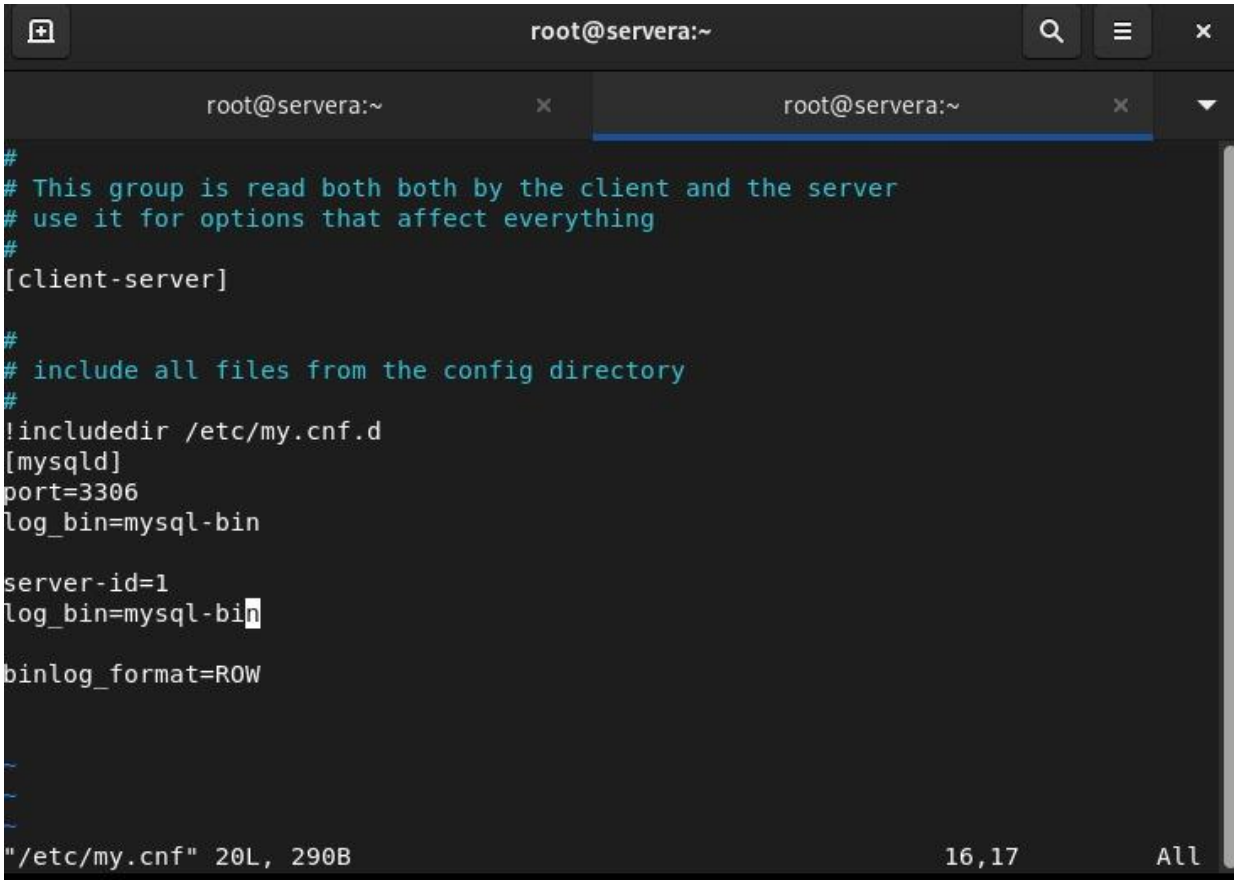
The vulnerability in the application that permitted SQL injection attacks made it possible for attackers to tamper with queries that would allow them to bypass authentication through SQL input.

In order to counteract the problem, the Principle of Least Privilege was incorporated into the application to ensure that only a limited MySQL account is used. Other security measures include password encryption.

ILO 5 — HIGH AVAILABILITY AND READ SCALABILITY IMPLEMENTATION

MASTER SERVER CONFIGURATION

My.cnf Configurations



MySQL server primary setup involved editing of the my.cnf file with settings for enabling the use of binary logging and setting the server ID. Binary logging ensures that all the activities of the database are logged and subsequently replicated to the replica server. The setting of the server ID makes it possible to distinguish the primary server from others.

Master Status

```
mysql> SHOW MASTER STATUS;
+------------------+----------+--------------+------------------+---------------
----+
| File             | Position | Binlog_Do_DB | Binlog_Ignore_DB | Executed_Gtid_
Set |
+------------------+----------+--------------+------------------+---------------
----+
| mysql-bin.000001 |      866 |              |                  |
    |
+------------------+----------+--------------+------------------+---------------
```

SHOW MASTER STATUS command was issued to get the value of binary log filename and log position. These parameters are important when configuring MySQL replication. Since these values indicate the exact point where data should be synchronized. The use of this command showed that binary logging was working properly on the primary database server.

Replication User Configuration

```
mysql> GRANT REPLICATION SLAVE ON *.* TO 'test_user'@'%';
Query OK, 0 rows affected (0.00 sec)

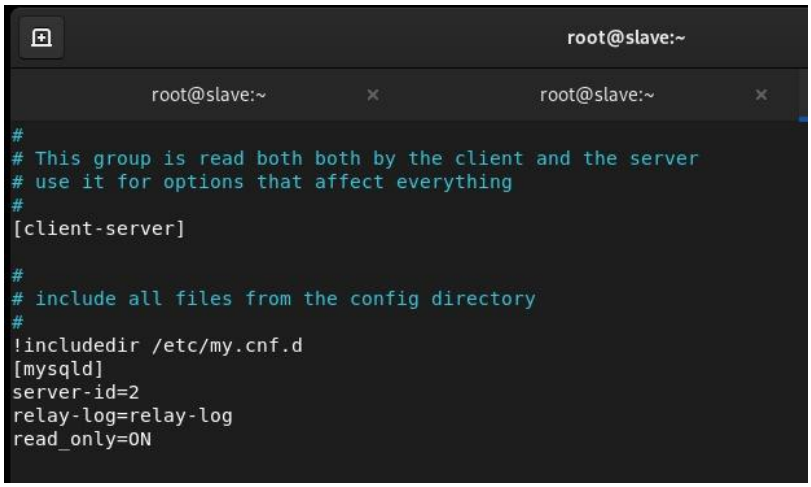
mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.00 sec)

mysql> FLUSH TABLES WITH READ LOCK;
Query OK, 0 rows affected (0.00 sec)
```

A dedicated replication user was created for transferring binary log events from the master server to the slave server. In order to do this, a replication user was assigned REPLICATION SLAVE privileges on the server. As such, the privilege separation helps improve the security level of databases.

SLAVE SERVER CONFIGURATION

My.cnf file



In addition to that, a dedicated replication user was created for transferring binary log events from the master server to the slave server. In order to do this, a replication user was assigned REPLICATION SLAVE privileges on the server. As such, the privilege separation helps improve the security level of databases.

Replication Commands

```
ysql> CHANGE MASTER TO
    -> MASTER_HOST='192.168.16.19',
    -> MASTER_USER='test_user',
    -> MASTER_PASSWORD='Test@123',
    ->  MASTER_LOG_FILE='mysql-bin.000001',
    -> MASTER_LOG_POS=866;
uery OK, 0 rows affected, 8 warnings (0.01 sec)
```

```
mysql> START SLAVE;
Query OK, 0 rows affected, 1 warning (0.03 sec)

mysql> SHOW SLAVE STATUS\G
*************************** 1. row ***************************
               Slave_IO_State: Waiting for source to send event
                  Master_Host: 192.168.16.19
                  Master_User: test_user
                  Master_Port: 3306
                Connect_Retry: 60
              Master_Log_File: mysql-bin.000001
          Read_Master_Log_Pos: 866
               Relay_Log_File: relay-log.000002
                Relay_Log_Pos: 326
        Relay_Master_Log_File: mysql-bin.000001
             Slave_IO_Running: Yes
            Slave_SQL_Running: Yes
              Replicate_Do_DB:
```

CHANGE MASTER TO command was issued in order to create a configuration for replicating data received from the master server. It was necessary to specify the connection information such as master server id, replication user, password, master log file name, and log position. Then START SLAVE command was used to start the replication process successfully.

Status Check

SHOW SLAVE STATUS\G command allowed us to view the results of MySQL replication operation. We have obtained the Slave_IO_Running and Slave_SQL_Running values, which both were set to Yes. It indicated the correct functioning of MySQL replication.

```
mysql> START SLAVE;
Query OK, 0 rows affected, 1 warning (0.03 sec)

mysql> SHOW SLAVE STATUS\G
*************************** 1. row ***************************
               Slave_IO_State: Waiting for source to send event
                  Master_Host: 192.168.16.19
                  Master_User: test_user
                  Master_Port: 3306
                Connect_Retry: 60
              Master_Log_File: mysql-bin.000001
          Read_Master_Log_Pos: 866
               Relay_Log_File: relay-log.000002
                Relay_Log_Pos: 326
        Relay_Master_Log_File: mysql-bin.000001
             Slave_IO_Running: Yes
            Slave_SQL_Running: Yes
              Replicate_Do_DB:
```

Replication Testing Connectivity

```
[root@slave ~]# telnet 192.168.16.19 3306
Trying 192.168.16.19...
Connected to 192.168.16.19.
Escape character is '^]'.
```

We have created new databases and new tables on the primary server, and then checked out if they would appear on the replica server. The results showed successful replication, so we can say that the network and server configuration is good.

HAPROXY CONFIGURATION

Definition:

HAProxy (High Availability Proxy) is a free, very fast and reliable solution that provides high availability and load balancing for TCP and HTTP-based applications by distributing client requests among several servers.

HAProxy Configuration file

```
[root@haproxy ~]# vim /etc/haproxy/haproxy.cfg
```

Defaults Configurations

```
defaults
    mode                    tcp
    log                     global
    option                  tcplog
    option                  dontlognull
    option                  redispatch
    retries                 3
    timeout http-request    10s
    timeout queue           1m
    timeout connect         10s
    timeout client          1m
    timeout server          1m
    timeout http-keep-alive 10s
    timeout check           10s
    maxconn                 3000
```

Main configurations

```
# main frontend which proxys to the backends
#---------------------------------------------------------------------
frontend mysql_front
    mode tcp
    bind *:3307
    default_backend mysql_back

backend mysql_back
    mode tcp
    balance roundrobin
    option tcp-check

    server mysql1 192.168.16.19:3306 check
    server mysql2 192.168.16.22:3306 check

listen stats
    bind *:8080
    mode http
    stats enable
    stats uri /stats
    stats refresh 10s
    stats auth admin:admin123
```

Explanation:

The statistics panel for HAProxy presents live monitoring data on the condition of the backend server, connection, and load-balancing process. The statistics panel was accessed via port 8080, with username/password authentication.

Connectivity Testing

```
[root@haproxy ~]# mysql -u haproxy -p -h 192.168.16.22
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 14
Server version: 8.0.45 Source distribution

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> exit
```

Load Balancing Testing

```
mysql> SELECT @@hostname;
+---------------------+
| @@hostname          |
+---------------------+
| servera.example.com |
+---------------------+
1 row in set (0.01 sec)
```

SELECT @@hostname was used after establishing connection via HAProxy in order to check on which of the backend MySQL servers the request was served. Multiple logins resulted in the selection of different backend servers.

HAProxy's Benefits for This Task

- Higher availability of the database
- Enhanced scalability for reading operations
- Less workload on servers
- Load balancing between the backend servers
- Constant running of services in case of failures
- Accessing databases from one single point

ILO 6 — INDEPENDENT DATABASE SECURITY AUDIT AND COMPLIANCE REVIEW

Audit-Overview

For the sake of spotting the vulnerabilities that can be exploited an audit on the security of the database was performed to analyze the configuration settings on the MySQL instance implemented, find misconfigurations, check access control and investigate compliance with the security best practices on security.

Audit Scope

Below was audited in the system:

- User privilege management
- Authentication controls
- Firewall settings
- Open network ports
- Replication security
- HAProxy exposure
- Backup system
- Access control mechanism
- Password policy
- Logging settings of database

Compliance Standard — ISO 27001

Definition:

ISO 27001 is an international information security standard used to manage and protect organizational information assets through risk-based security controls.

AUDIT FINDING TABLES

Control Area	Status	Risk Level	Observation	Recommendation
Firewall port 3306	Medium-Risk	Medium	Port 3306 was opened out of replication purposes and due to remote connectivity	Access should be restricted with the usage of specific Ip address instead of allowing unauthorized access
HAProxy Port 3307	Acceptable-level	Low	Port for the 3307 was used for the load-balancing	Limit the access to trusted authorities like internal clients only
Bind-address=0.0.0.0	High-Risk	High	To accept remote-connection from all net interfaces	Limit to trusted-host only
SELinux Disabled	High-Risk	High	Was disabled with the usage of setenforce 0	Maintain SElinux getenforce status at enforcing
Password Validation	Secured	High	password validation component was enabled	keep maintaining and enforcement of strong-level passwords
Anonymous Users	Secured	Low	those anonymous account was removed	no particular action required
SQL Injection Protection	secured	Low	It was shown successful synchronization	Carry on with secure coding practices as usual

Firewall Configuration Review

```
[root@servera ~]# firewall-cmd --add-port=3306/tcp --permanent
success
[root@servera ~]# firewall-cmd --reload
success
```

The firewall was configured to permit MySQL replication, HAProxy database routing, and access to the HAProxy dashboard. Although essential for functionality, this poses a greater network risk and should be limited to internal use only.

SELinux Security Observation

```
[root@servera ~]# setenforce 0
```

As such, in order to facilitate easy communication between HAProxy and MySQL for the practical implementation, the SELinux was disabled. This practice is not encouraged since disabling the SELinux weakens the system level protection of the system.

Replication Security Review

```
mysql> CREATE USER 'staff_user'@'%' IDENTIFIED WITH mysql_native_password BY 'Staff@123';
Query OK, 0 rows affected (0.00 sec)
```

Risk

Wildcards being used as host values might result in replication credentials becoming available to unauthorized hosts.

Recommendation

Replication should be allowed only from the IP address of the replica server.

Overall Audit Summary

The MySQL environment was successfully shown to be capable of exhibiting authentication security, replication, load balancing, access control policies, and hardening of the database. There were also medium and high risk discoveries, mostly because of the practicality in the laboratory setting such as open ports, disabling SELinux, and enabling remote access from anywhere.

FINAL-EXPLORATION

In conclusion, it should be emphasized that the project has succeeded in creating a high-quality and reliable MySQL database system for a company working in the financial sector. Consequently, it can be considered an example of designing a secure database, authentications, access control, threats of databases and ways of solving them, replication, high availability architecture, and database security audits.

First of all, in order to establish a secure database system, it is necessary to design a database environment containing various interconnected tables with the company information, such as data on clients, employees, transactions, authentication, logs, and backups. The maintenance of the CIA principle in a database was ensured by authentication procedures, restricted privileges, relations of primary keys and foreign keys, validation, backing up, replication, and architecture of HAProxy ensuring high availability.

Secondly, the process of authenticating and gaining access to database data was secured through authentication plugins, privileged-separated user accounts in MySQL, role-based access control, and process of database hardening, including disabling anonymous users, enabling password validation policies, and not using insecure configuration parameters. The problem of SQL injection attacks was analyzed.

Finally, high availability and read scalability were provided using the MySQL Primary-Replica Replication technology and HAProxy Load Balancing. As far as the replication was concerned, databases functioned properly during its test, while the load balancing strategy ensured distribution of the load among back-end database servers using HAProxy.

Finally, a security audit of this database was conducted in accordance with ISO 27001 standards. As a result, the following strengths of database security were identified, while some problems in setting parameters could be mentioned, namely opened port numbers, disabling of SELinux, and remote access.

REFERENCES

[1] Oracle Corporation, MySQL 8.0 Reference Manual. [Online]. Available: https://dev.mysql.com/doc/

[2] Oracle Corporation, “MySQL Replication Configuration,” MySQL Documentation. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/replication.html?

[3] Oracle Corporation, “Authentication Plugins,” MySQL Documentation. [Online]. Available:

https://dev.mysql.com/doc/refman/8.0/en/authentication-plugins.html

[4] HAProxy Technologies, HAProxy Documentation. [Online]. Available: https://www.haproxy.org/

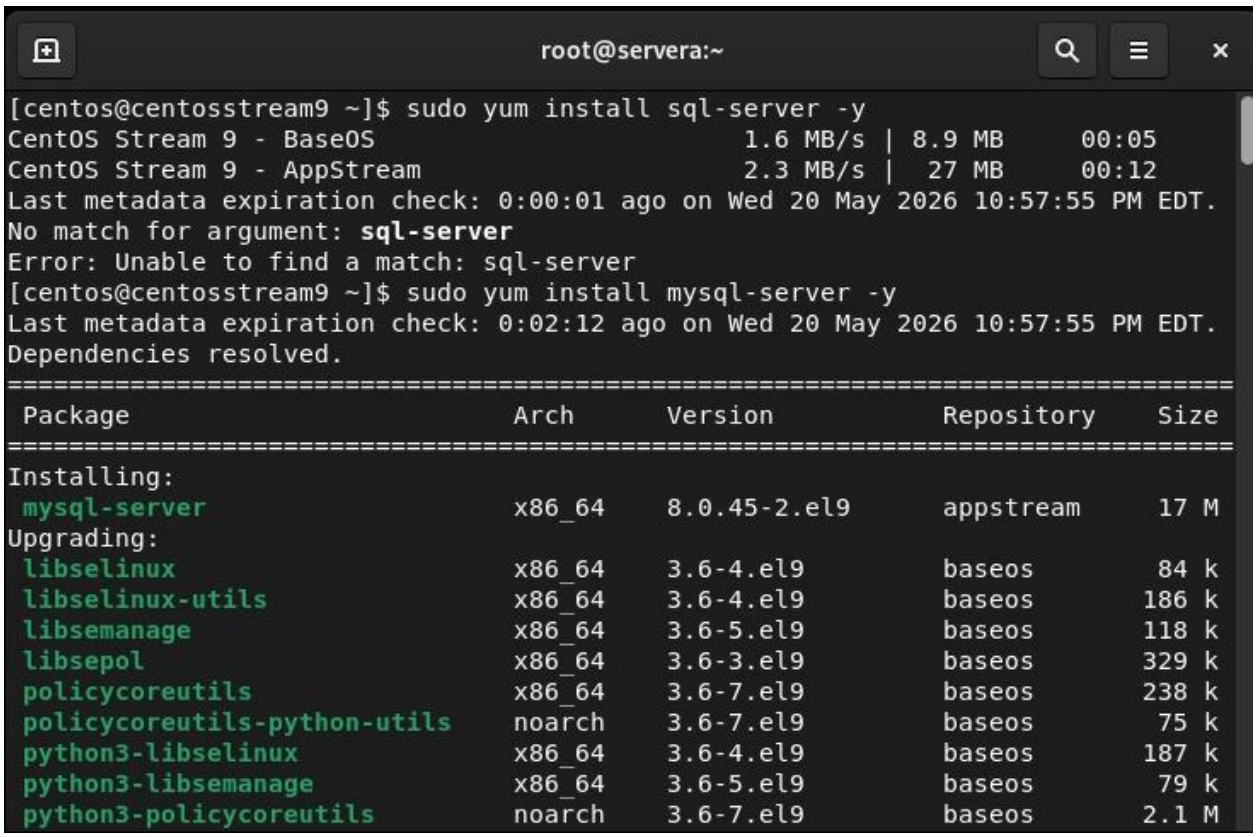
[5] International Organization for Standardization, ISO/IEC 27001 Information Security Management. [Online]. Available: https://www.iso.org/standard/27001?

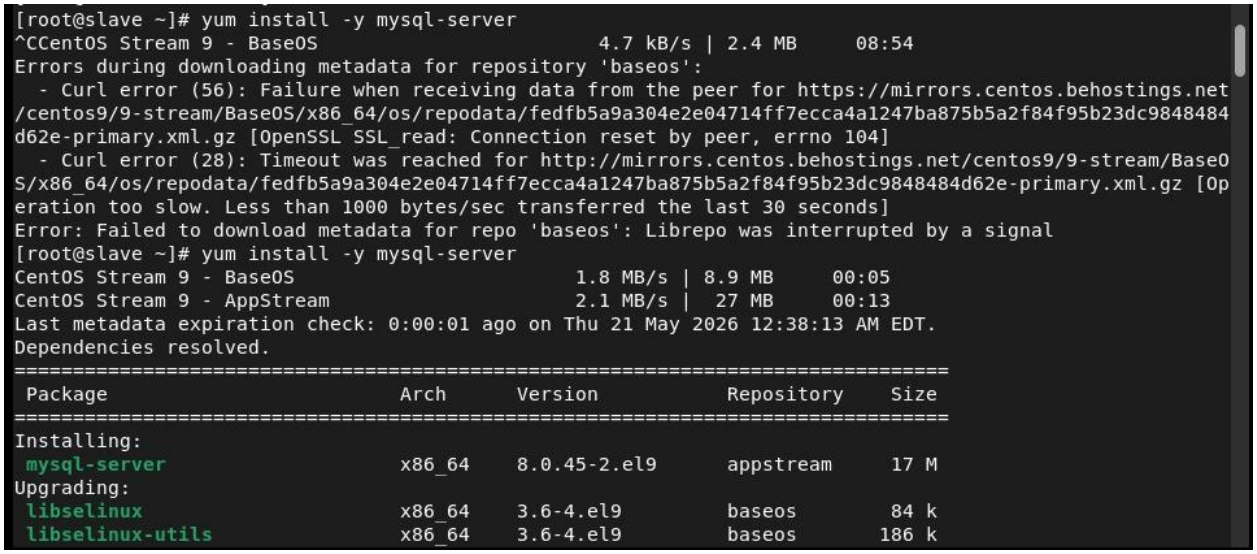
[6] OWASP Foundation, “SQL Injection Prevention,” OWASP Documentation. [Online]. Available: https://owasp.org/www-community/attacks/SQL_Injection

APPENDIX

Supporting Evidences Additional

Installations





```
[root@centosstream9 ~]# yum install -y haproxy
CentOS Stream 9 - BaseOS                        152 kB/s | 8.9 MB     01:00
CentOS Stream 9 - AppStream                     1.1 MB/s |  27 MB     00:25
Last metadata expiration check: 0:00:02 ago on Thu 21 May 2026 12:38:05 AM EDT.
Dependencies resolved.
================================================================================
 Package         Architecture   Version                 Repository         Size
================================================================================
Installing:
 haproxy         x86_64         2.8.14-3.el9            appstream         2.5 M

Transaction Summary
================================================================================
Install  1 Package

Total download size: 2.5 M
Installed size: 7.7 M
Downloading Packages:
haproxy-2.8.14-3.el9.x86_64.rpm                 985 kB/s | 2.5 MB     00:02
--------------------------------------------------------------------------------
Total                                           476 kB/s | 2.5 MB     00:05
CentOS Stream 9 - AppStream                     1.6 MB/s | 1.6 kB     00:00
Importing GPG key 0x8483C65D:
```

Service starts and status check

```
root@servera:~

Enter password for user root:
Error: Can't connect to local MySQL server through socket '/var/lib/mysql/mysql.
sock' (2)
[root@servera ~]# systemctl start mysqld
[root@servera ~]# systemctl enable mysqld
Created symlink /etc/systemd/system/multi-user.target.wants/mysqld.service → /us
r/lib/systemd/system/mysqld.service.
[root@servera ~]# systemctl status mysqld
● mysqld.service - MySQL 8.0 database server
     Loaded: loaded (/usr/lib/systemd/system/mysqld.service; enabled; vendor pr>
     Active: active (running) since Wed 2026-05-20 23:56:58 EDT; 16s ago
   Main PID: 34421 (mysqld)
     Status: "Server is operational"
      Tasks: 38 (limit: 10921)
     Memory: 418.4M
        CPU: 4.603s
     CGroup: /system.slice/mysqld.service
             └─34421 /usr/libexec/mysqld --basedir=/usr

May 20 23:56:52 servera.example.com systemd[1]: Starting MySQL 8.0 database ser>
May 20 23:56:52 servera.example.com mysql-prepare-db-dir[34347]: Initializing M>
May 20 23:56:58 servera.example.com systemd[1]: Started MySQL 8.0 database serv>
[root@servera ~]# mysql_secure_installation
```

```
[root@slave ~]# systemctl start mysqld
[root@slave ~]# systemctl enable mysqld
Created symlink /etc/systemd/system/multi-user.target.wants/mysqld.service → /usr/lib/systemd/system/mys
qld.service.
[root@slave ~]# systemctl status mysqld
● mysqld.service - MySQL 8.0 database server
     Loaded: loaded (/usr/lib/systemd/system/mysqld.service; enabled; vendor pr>
     Active: active (running) since Thu 2026-05-21 00:42:34 EDT; 17s ago
   Main PID: 34182 (mysqld)
     Status: "Server is operational"
      Tasks: 38 (limit: 10921)
     Memory: 420.9M
        CPU: 2.505s
     CGroup: /system.slice/mysqld.service
             └─34182 /usr/libexec/mysqld --basedir=/usr
```

```
[root@haproxy ~]# systemctl start haproxy
[root@haproxy ~]# systemctl enable haproxy
Created symlink /etc/systemd/system/multi-user.target.wants/haproxy.service → /u
sr/lib/systemd/system/haproxy.service.
[root@haproxy ~]# systemctl status haproxy
● haproxy.service - HAProxy Load Balancer
     Loaded: loaded (/usr/lib/systemd/system/haproxy.service; enabled; vendor p>
     Active: active (running) since Thu 2026-05-21 00:49:39 EDT; 22min ago
   Main PID: 32039 (haproxy)
     Status: "Ready."
      Tasks: 3 (limit: 10921)
     Memory: 11.8M
        CPU: 499ms
     CGroup: /system.slice/haproxy.service
             ├─32039 /usr/sbin/haproxy -Ws -f /etc/haproxy/haproxy.cfg -f /etc/>
             └─32041 /usr/sbin/haproxy -Ws -f /etc/haproxy/haproxy.cfg -f /etc/>
```

HAPROXY Connectivity Check Via IPs

```
[root@haproxy ~]# mysql -u haproxy -p -h 192.168.16.19
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 10
Server version: 8.0.45 Source distribution

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> exit
```

```
[root@haproxy ~]# mysql -u haproxy -p -h 192.168.16.22
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 14
Server version: 8.0.45 Source distribution

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> exit
```

Database Data Insertion

```
mysql> INSERT INTO backup_records (backup_name, classification_level, backup_date, created_by) VALUES
    -> ('full_backup_01', 'Internal', '2026-05-01', 1),
    -> ('full_backup_02', 'Internal', '2026-05-02', 1),
    -> ('incremental_01', 'Internal', '2026-05-03', 5),
    -> ('incremental_02', 'Internal', '2026-05-04', 5),
    -> ('differential_01', 'Internal', '2026-05-05', 1);
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

```
mysql> INSERT INTO audit_logs (user_id, action, classification_level, log_date) VALUES
    -> (1, 'Login Success', 'Internal', '2026-05-01'),
    -> (2, 'Viewed Customers', 'Internal', '2026-05-02'),
    -> (3, 'Updated Employee Data', 'Internal', '2026-05-03'),
    -> (4, 'Generated Report', 'Internal', '2026-05-04'),
    -> (5, 'Database Backup Taken', 'Internal', '2026-05-05');
Query OK, 5 rows affected (0.00 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

```
mysql> INSERT INTO transactions (customer_id, amount, classification_level, transaction_date) VALUES
    -> (1, 5000.00, 'Restricted', '2026-05-01'),
    -> (2, 12000.00, 'Restricted', '2026-05-02'),
    -> (3, 7500.00, 'Restricted', '2026-05-03'),
    -> (4, 20000.00, 'Restricted', '2026-05-04'),
    -> (5, 3000.00, 'Restricted', '2026-05-05');
Query OK, 5 rows affected (0.00 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

```
mysql> INSERT INTO transactions (customer_id, amount, classification_level, transaction_date) VALUES
    -> (1, 5000.00, 'Restricted', '2026-05-01'),
    -> (2, 12000.00, 'Restricted', '2026-05-02'),
    -> (3, 7500.00, 'Restricted', '2026-05-03'),
    -> (4, 20000.00, 'Restricted', '2026-05-04'),
    -> (5, 3000.00, 'Restricted', '2026-05-05');
Query OK, 5 rows affected (0.00 sec)
Records: 5  Duplicates: 0  Warnings: 0

Query OK, 5 rows affected (0.00 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

```
mysql> INSERT INTO employees (user_id, full_name, national_id, salary, classification_level) VALUES
    -> (1, 'John Perera', 'NIC001', 150000.00, 'Restricted'),
    -> (2, 'Kamal Silva', 'NIC002', 90000.00, 'Confidential'),
    -> (3, 'Nimal Fernando', 'NIC003', 85000.00, 'Confidential'),
    -> (4, 'Saman Kumara', 'NIC004', 95000.00, 'Confidential'),
    -> (5, 'Ravi Jayasinghe', 'NIC005', 80000.00, 'Internal');
Query OK, 5 rows affected (0.00 sec)
Records: 5  Duplicates: 0  Warnings: 0
```


```
mysql> INSERT INTO employees (user_id, full_name, national_id, salary, classification_level) VALUES
    -> (1, 'John Perera', 'NIC001', 150000.00, 'Restricted'),
    -> (2, 'Kamal Silva', 'NIC002', 90000.00, 'Confidential'),
    -> (3, 'Nimal Fernando', 'NIC003', 85000.00, 'Confidential'),
    -> (4, 'Saman Kumara', 'NIC004', 95000.00, 'Confidential'),
    -> (5, 'Ravi Jayasinghe', 'NIC005', 80000.00, 'Internal');
Query OK, 5 rows affected (0.00 sec)
Records: 5  Duplicates: 0  Warnings: 0
```


Queries

```
mysql> SELECT user, host FROM mysql.user;
+------------------+-----------+
| user             | host      |
+------------------+-----------+
| admin_user       | %         |
| auditor_user     | %         |
| backup_user      | %         |
| haproxy          | %         |
| staff_user       | %         |
| test_user        | %         |
| mysql.infoschema | localhost |
| mysql.session    | localhost |
| mysql.sys        | localhost |
| root             | localhost |
+------------------+-----------+
10 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM employees;
+-------------+---------+----------------+-------------+-----------+----------------------+
| employee_id | user_id | full_name      | national_id | salary    | classification_level |
+-------------+---------+----------------+-------------+-----------+----------------------+
|           1 |       1 | John Perera    | NIC001      | 150000.00 | Restricted           |
|           2 |       2 | Kamal Silva    | NIC002      |  90000.00 | Confidential         |
|           3 |       3 | Nimal Fernando | NIC003      |  85000.00 | Confidential         |
|           4 |       4 | Saman Kumara   | NIC004      |  95000.00 | Confidential         |
|           5 |       5 | Ravi Jayasinghe | NIC005     |  80000.00 | Internal             |
+-------------+---------+----------------+-------------+-----------+----------------------+
5 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM users;
+---------+----------+-----------+---------+---------------------+
| user_id | username | password  | role    | created_at          |
+---------+----------+-----------+---------+---------------------+
|       1 | admin1   | admin123  | admin   | 2026-05-01 00:00:00 |
|       2 | staff1   | staff123  | staff   | 2026-05-02 00:00:00 |
|       3 | staff2   | staff456  | staff   | 2026-05-03 00:00:00 |
|       4 | auditor1 | audit123  | auditor | 2026-05-04 00:00:00 |
|       5 | backup1  | backup123 | backup  | 2026-05-05 00:00:00 |
+---------+----------+-----------+---------+---------------------+
5 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM transactions;
+----------------+-------------+----------+----------------------+------------------+
| transaction_id | customer_id | amount   | classification_level | transaction_date |
+----------------+-------------+----------+----------------------+------------------+
|              1 |           1 |  5000.00 | Restricted           | 2026-05-01       |
|              2 |           2 | 12000.00 | Restricted           | 2026-05-02       |
|              3 |           3 |  7500.00 | Restricted           | 2026-05-03       |
|              4 |           4 | 20000.00 | Restricted           | 2026-05-04       |
|              5 |           5 |  3000.00 | Restricted           | 2026-05-05       |
+----------------+-------------+----------+----------------------+------------------+
5 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM audit_logs;
+--------+---------+-----------------------+----------------------+------------+
| log_id | user_id | action                | classification_level | log_date   |
+--------+---------+-----------------------+----------------------+------------+
|      1 |       1 | Login Success         | Internal             | 2026-05-01 |
|      2 |       2 | Viewed Customers      | Internal             | 2026-05-02 |
|      3 |       3 | Updated Employee Data | Internal             | 2026-05-03 |
|      4 |       4 | Generated Report      | Internal             | 2026-05-04 |
|      5 |       5 | Database Backup Taken | Internal             | 2026-05-05 |
+--------+---------+-----------------------+----------------------+------------+
5 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM backup_records;
+-----------+-----------------+----------------------+-------------+------------+
| backup_id | backup_name     | classification_level | backup_date | created_by |
+-----------+-----------------+----------------------+-------------+------------+
|         1 | full_backup_01  | Internal             | 2026-05-01  |          1 |
|         2 | full_backup_02  | Internal             | 2026-05-02  |          1 |
|         3 | incremental_01  | Internal             | 2026-05-03  |          5 |
|         4 | incremental_02  | Internal             | 2026-05-04  |          5 |
|         5 | differential_01 | Internal             | 2026-05-05  |          1 |
+-----------+-----------------+----------------------+-------------+------------+
5 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM customers;
+-------------+-----------------+-------------+--------------------+------------+----------------------+
| customer_id | full_name       | national_id | email              | phone      | classification_level |
+-------------+-----------------+-------------+--------------------+------------+----------------------+
|           1 | Amal Perera     | CUST001     | amal@gmail.com     | 0771111111 | Confidential         |
|           2 | Nimal Dias      | CUST002     | nimal@gmail.com    | 0772222222 | Confidential         |
|           3 | Kasun Silva     | CUST003     | kasun@gmail.com    | 0773333333 | Confidential         |
|           4 | Sanduni Fernando | CUST004    | sanduni@gmail.com  | 0774444444 | Confidential         |
|           5 | Tharindu Jay    | CUST005     | tharindu@gmail.com | 0775555555 | Confidential         |
+-------------+-----------------+-------------+--------------------+------------+----------------------+
5 rows in set (0.01 sec)
```

Creation of Test User for testing purposes

```
mysql> USE enterprise_secure_db
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> INSERT INTO customers (full_name, national_id, email, phone)
    -> VALUES ('Test User', 'CUST999', 'test@gmail.com', '0771234567');
Query OK, 1 row affected (0.01 sec)
```